Solution 1: Hospital Lab Integration

Pattern: Adapter | Resolves interface mismatches between incompatible lab APIs and the hospital system without modifying existing lab classes.

```
// Target Interface
interface PatientReport {
    void getPatientReport();
}

// Existing Lab APIs (Adaptees)
class LabA {
    void getBloodReport() {
        System.out.println("LabA: Blood report fetched");
    }
}

class LabB {
    void fetchLabResults() {
        System.out.println("LabB: Lab results fetched");
    }
}

// Adapter
class LabAdapter implements PatientReport {
    private LabA labA;
    private LabB labB;

    LabAdapter(String labType) {
        if (labType.equalsIgnoreCase("LabA")) {
            labA = new LabA();
        } else if (labType.equalsIgnoreCase("LabB")) {
            labB = new LabB();
        }
    }

    public void getPatientReport() {
        if (labA != null) {
            labA.getBloodReport();
        } else if (labB != null) {
            labB.fetchLabResults();
        }
    }
}

// Client
public class HospitalAdapterDemo {
    public static void main(String[] args) {
        PatientReport report1 = new LabAdapter("LabA");
        report1.getPatientReport();

        PatientReport report2 = new LabAdapter("LabB");
        report2.getPatientReport();
    }
}
```

Solution 2: Video Streaming Platform

Pattern: Proxy (Virtual/Caching) | Delays expensive video loading until first play and reuses the loaded instance on subsequent calls.

```
// Subject Interface
interface Video {
    void play();
}
```

```
// Real Object
class RealVideo implements Video {
    private String name;
    RealVideo(String name) {
        this.name = name;
        loadVideo();
    }
    private void loadVideo() {
        System.out.println("Loading video: " + name);
    }
    public void play() {
        System.out.println("Playing video: " + name);
    }
}

// Proxy
class VideoProxy implements Video {
    private RealVideo realVideo;
    private String name;

    VideoProxy(String name) { this.name = name; }

    public void play() {
        if (realVideo == null) {
            realVideo = new RealVideo(name); // lazy loading
        }
        realVideo.play();
    }
}

// Client
public class VideoProxyDemo {
    public static void main(String[] args) {
        Video video = new VideoProxy("movie.mp4");
        video.play(); // loads + plays
        video.play(); // only plays (no reload)
    }
}
```

Solution 3: Gaming Application

Pattern: Flyweight | Shares intrinsic state (type, texture, color) across thousands of objects; only extrinsic state (position) is passed at render time.

```
import java.util.HashMap;

// Flyweight
class GameObject {
    private String type;
    private String texture;
    private String color;

    GameObject(String type, String texture, String color) {
        this.type    = type;
        this.texture = texture;
        this.color   = color;
    }

    void display(int x, int y) {
        System.out.println(type + " displayed at (" + x + "," + y + ")");
    }
}

// Flyweight Factory
```

```
class GameObjectFactory {
    private static HashMap<String, GameObject> objects = new HashMap<>();

    static GameObject getObject(String type) {
        if (!objects.containsKey(type)) {
            objects.put(type, new GameObject(type, "DefaultTexture", "Green"));
        }
        return objects.get(type);
    }
}

// Client
public class FlyweightGameDemo {
    public static void main(String[] args) {
        GameObject tree1 = GameObjectFactory.getObject("Tree");
        tree1.display(10, 20);

        GameObject tree2 = GameObjectFactory.getObject("Tree");
        tree2.display(30, 40); // reused object

        GameObject rock = GameObjectFactory.getObject("Rock");
        rock.display(50, 60);
    }
}
```

Solution 4: E-Commerce Platform

Pattern: Facade | Wraps complex subsystem calls behind a single completePurchase() method, reducing client complexity.

```
// Subsystems
class LoginService {
    void login() { System.out.println("User logged in"); }
}

class ProductService {
    void selectProduct() { System.out.println("Product selected"); }
}

class PaymentService {
    void makePayment() { System.out.println("Payment successful"); }
}

class OrderService {
    void trackOrder() { System.out.println("Order tracking started"); }
}

// Facade
class EcommerceFacade {
    private LoginService   login   = new LoginService();
    private ProductService product = new ProductService();
    private PaymentService payment = new PaymentService();
    private OrderService   order   = new OrderService();

    void completePurchase() {
        login.login();
        product.selectProduct();
        payment.makePayment();
        order.trackOrder();
    }
}

// Client
public class FacadeEcommerceDemo {
```

```
    public static void main(String[] args) {
        EcommerceFacade facade = new EcommerceFacade();
        facade.completePurchase();
    }
}
```